CHAPTER 4

BASIC COMPUTER ORGANIZATION AND DESIGN

- ALU configuration
- Classification of ISA
- Instruction format and code
- Organization and Design of AC machine
  - Stored program organization in AC machine
  - Types of instruction in AC machine
  - Timing and control unit operation of AC machine
  - Instruction cycle
  - summery

4. BASIC COMPUTER ORG. AND DESIGN

Q What to organize?

Q How to organize?

Q Where to start ?

Ans: What to organize?

ALU with other CPU components.

Ans: How to organize?

For efficient design and operation of a digital computer, its basic component ALU and external components like process registers (of CPU process memory) storage memory and other function registers should have known configuration which we mean organization so as to have a complete control over micro operations.

The way how ALU organized with other CPU components decides the instruction format.

Ans: Where to start ?

Design of CPU starts from choosing ALU configuration scheme because the speed of execution and the resource that we use decided by the architecture of the configuration. Different CPU of Different architecture have different characteristics like speed, execution phase, operation code, instruction code, instruction size, instruction format, instruction density, memory traffic, addressing mode etc. The engineering work here is called **Instruction Set Architecture ISA.**

ALU configuration

Once an ALU is designed, we need to define how it interacts with the rest of the processor components.

- There are different ways of ALU configurations which we call **ISA,** We can choose any one, all ISA have advantages and disadvantages.
- The instruction sets can be differentiated by
  - ✓ Operand storage in the CPU
  - ✓ Number of explicit operands per instruction
  - ✓ Operand location
  - ✓ Operations
  - ✓ Type and size of operands

Internal storage of CPU

The type of **internal storage** in the CPU is the most basic differentiation. The major choices are

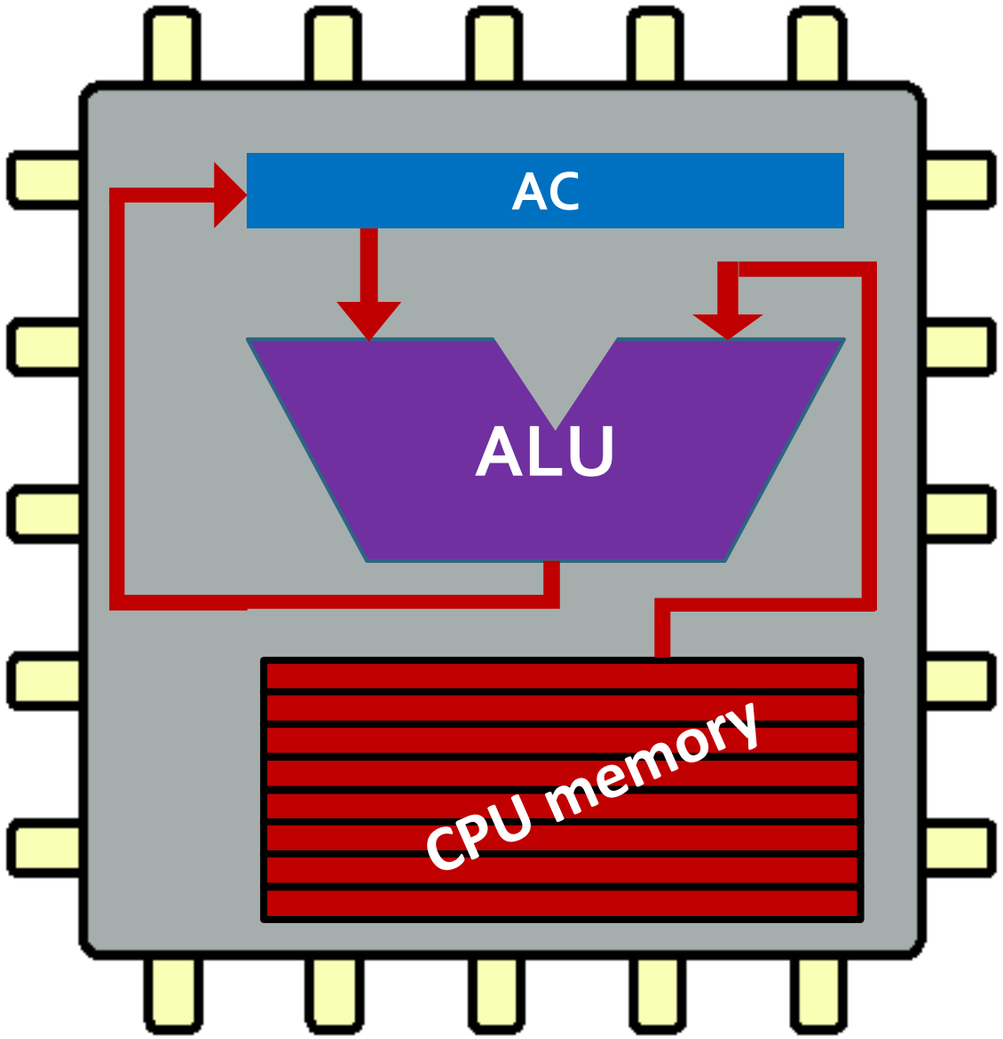
- ✓ A stack (the operands are implicitly on top of the stack)
- ✓ An accumulator (one operand is implicitly the accumulator)
- ✓ A set of registers (all operands are explicit either registers or memory locations)

The code segment example C = A + B how it would appear on the classes of instruction sets

Stack	**Accumulator**	**Register**
PUSH A	Load A	Load R1,A
PUSH B	ADD B	ADD R1,B
ADD	Store C	Store C,R1
POP C		

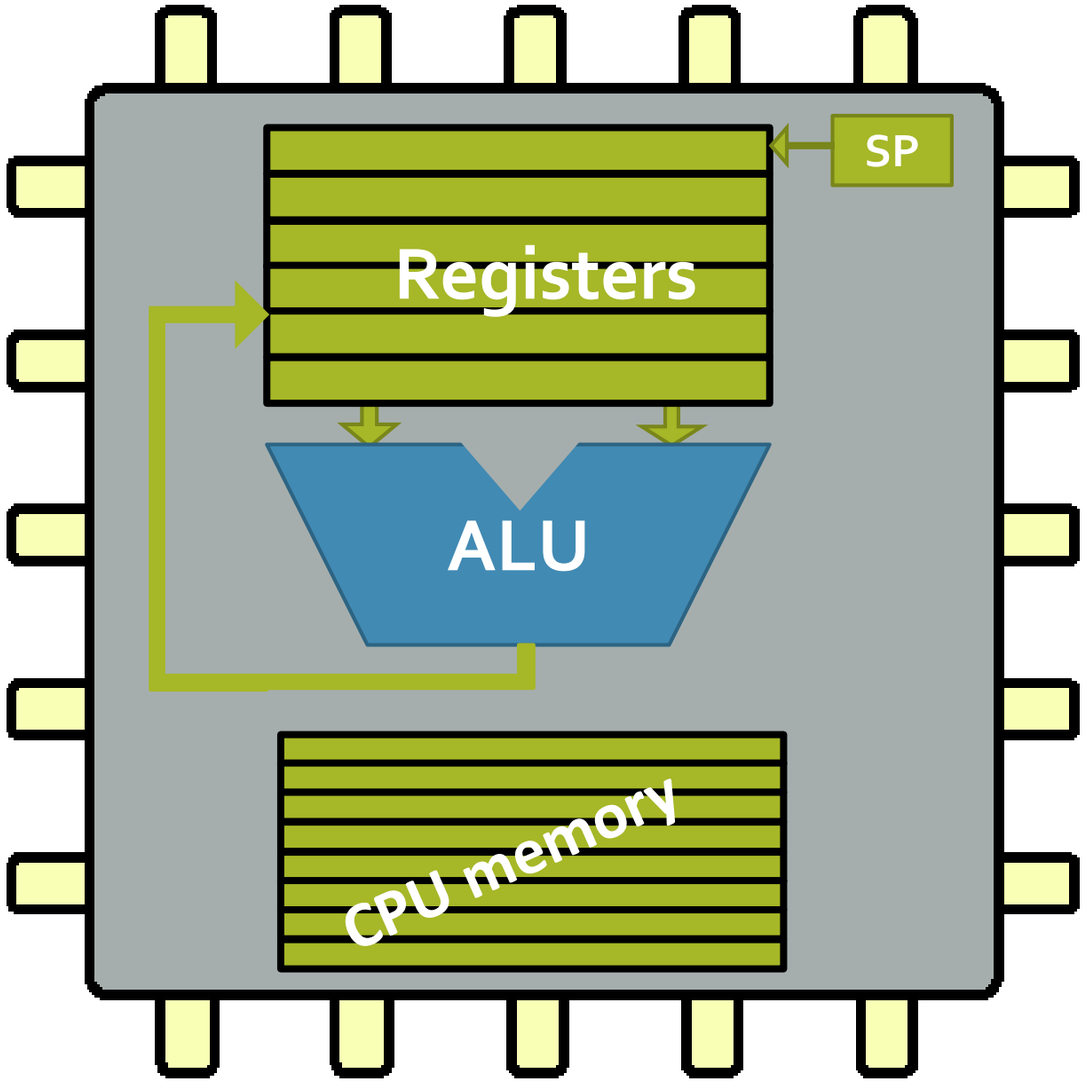
Accumulator Machine /AC-machine

- An accumulator machine has one special register, called the accumulator.
- The accumulator stores the result of every ALU operation, and is also one of the operands to every instruction. This means that our ISA can be less complicated, because instructions only need to specify one operand, instead of two operands and a destination.
- Accumulator architectures have simple ISAs and are typically very fast, but additional software needs to be written to load the accumulator with proper values.
- We study this organization and its instruction set in this course.



Register Stack Machine

- A register stack is like a combination of the Register- to-Register and the accumulator structures. In a register stack, the ALU reads the operands from the top of the stack, and the result is pushed onto the top of the stack. Complicated mathematical operations require decomposition into Reverse-Polish form, which can be difficult for programmers to use.
- However, many computer language compilers can produce reverse-polish notation easily because of the use of binary trees to represent instructions internally.



- Also, hardware needs to be created to implement the register stack, including **PUSH** and **POP** operations.
- additional hardware to detect and handle stack errors (pushing on a full stack, or popping an empty stack).
- The benefit is a highly simplified ISA.
- These machines are called "o-operand" or "zero address machines" because operands don't need to be specified, because all operations act on specified stack locations.
- In the diagram at right, "SP" is the pointer to the top of the stack. This is just one way to implement a stack structure, although it might be one of the easiest.

General Purpose Register Machines /GPR machine

There are two major instruction set characteristics that divide GPR architectures. They concern

- whether an ALU instruction has two or three operands how many of the operands may be memory addressed in ALU instruction.

ADD R3, R1, R2		ADD R1, R2
R3 ←R1 + R2	or	R1 ←R1 + R2

- Classification of General Purpose Register Machines GPR machine
  - ❖ Register- Register (Load/Store)

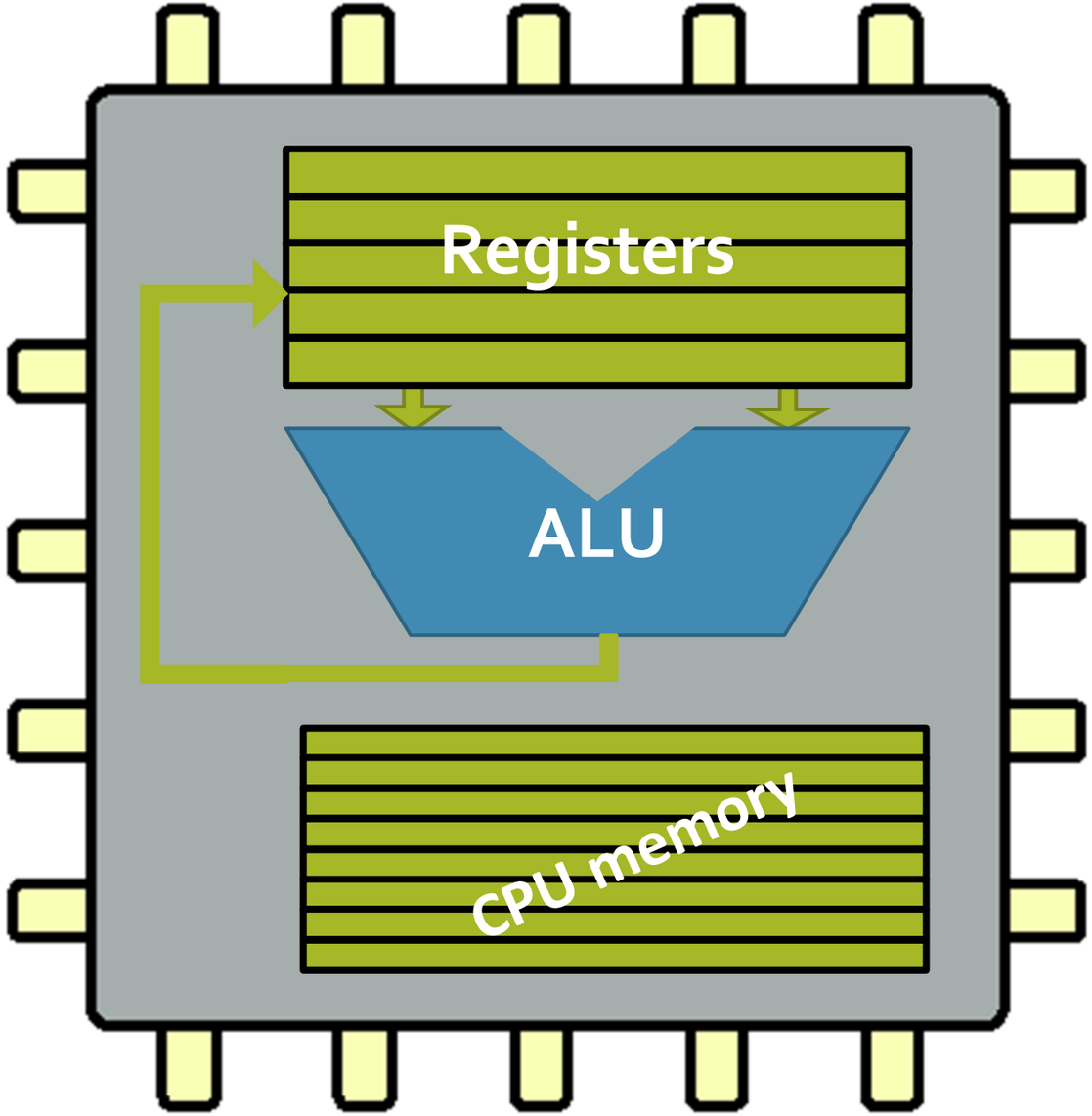
    ADD R3, R1, R2 (R3 ← R1 + R2)
  - ❖ Register - Memory

    ADD R1, A (R1 ← R1 + A)
  - ❖ Memory - Memory

    ADD C, A, B (C ← A + B)

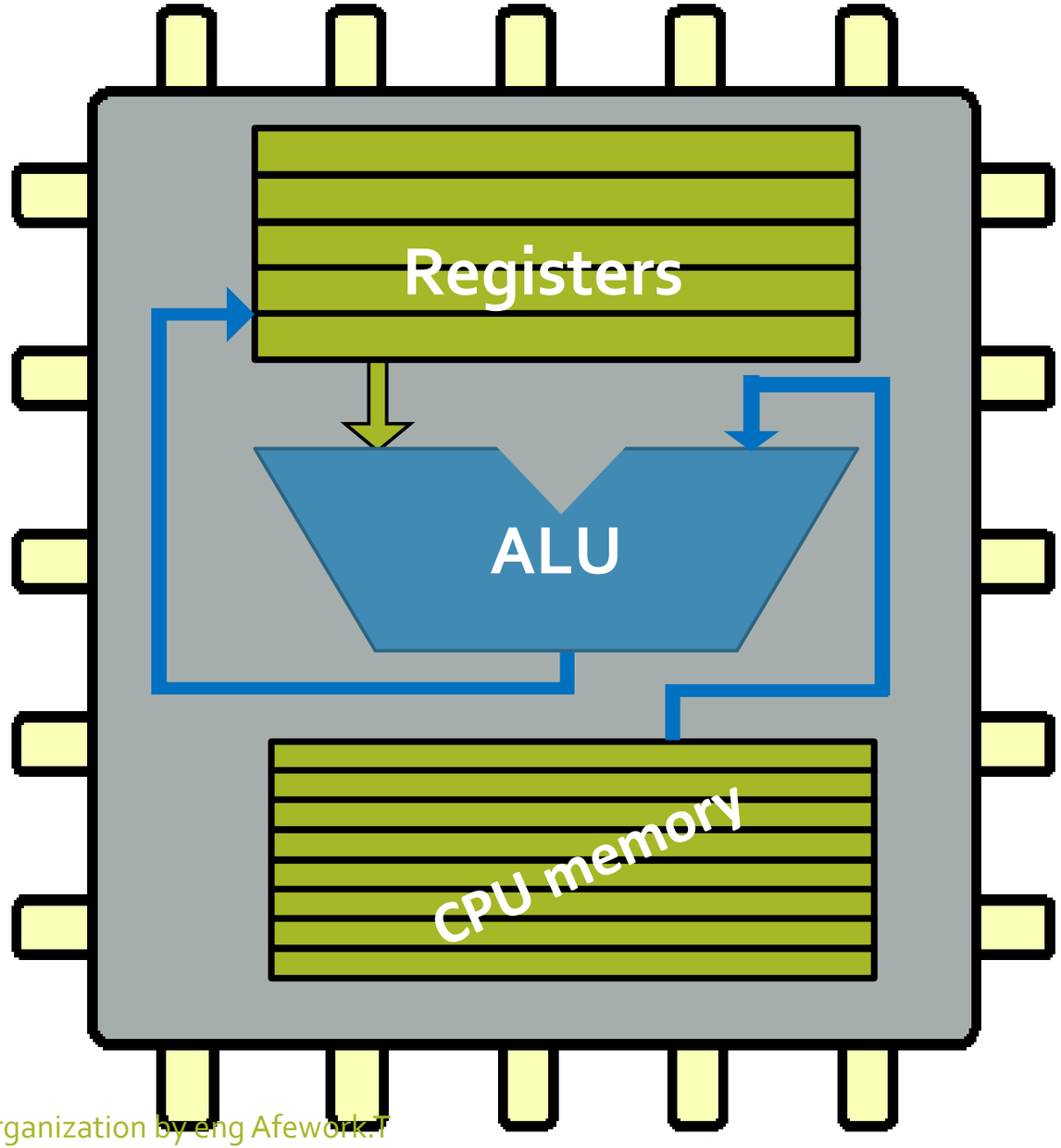
Register-to-Register ISA

- One of the more common architectures is a Register-to-register architecture, also called three address machine. In this configuration, the programmer can specify both source registers, and a destination register.
- Unfortunately, the ISA needs to be expanded to include fields for both source operands and the destination operands. This requires longer instruction word lengths.
- And it also requires additional effort (compared to the accumulator) to write results back to the register file after execution. This Write-back step can cause synchronization issues in pipelined processors.



Register-and-Memory ISA

- One complicated structure is a Register-and-Memory structure, like that shown at left. In this structure, one operand comes from a register file, and the other comes from external memory. In this structure, the ISA is complicated because each instruction word needs to be able to store a complete memory address, which can be very long. In practice, this scheme is not used directly, but is typically integrated into another scheme, such as a Register-to-Register scheme, for flexibility
- Some CISC architectures have the option of specifying one of the operands to an instruction as a memory address, although they are typically specified as a register address.



Comparing classes of ISA

Primary advantages and disadvantages of each class of machine		
Machine Type	Advantages	Disadvantages
Stack	Simple model of expression evaluation. Good code density.	A stack can't be randomly accessed. It makes it difficult to generate efficient code.
Accumulator	Minimizes internal state of machine. Short instructions	Since accumulator is only temporary storage, memory traffic is highest.
Register	Most general model for code generation	All operands must be named, leading to longer instructions.

INSTRUCTION FORMAT AND CODES

- ✓ The user of a computer can control the process by means of a program.
- ✓ A program is a set of instructions that specify the operations, operand, the sequence(control)
- ✓ An instruction is a binary code that specifies a sequence of micro operations.
- ✓ Instruction codes together with data are stored in memory(=Stored Program Concept)
- ✓ The computer reads each instruction from memory and places in a control register called **IR** instruction register.
- ✓ The control then interprets the binary code of the instruction and proceeds to execute it by issuing a sequence of micro operations.

Instruction Code :

- ✓ A group of bits that instruct the computer to perform a specific operation
- ✓ It is usually divided into parts



Instruction format

Instruction size: # of address line of CPU memory **+** n bit for 2^n distinct operation

- ✓ The length of instruction which means the size varies from CPU to CPU according to the ISA of CPU.

Operation code:

The most basic part of instruction code.

A group of n-bits that define 2^n distinct operations such as add,sub,mul,shift and complement and so on, are placed in this field.

Address : specify where the operand is found in the memory. This field can be divided in to parts according to the ISA of the machine. in some machines it has one operand address, two operand addresses or three operand addresses and there may be no need of addressing operand(zero address operand). The operands can be source operands or destination operand. Source operand can be one or two or no in the instruction.

A machine that can be designed from the combination of two or more ISA is set to have variable instruction length because of the number of operands used. These machines are called **CISC (**Complex Instruction Set Computers).They have large size insatruction to gether with small size instruction 1byte to 6 and 8 bytes. E.g. PIC microcontroller

Machines made to have reduced length of instructions to make uniform are named as **RISC** (Reduced Instruction Set Computers). E.g. AVR microcontroller

Both have advantages and disadvantages

According to address of operands in instruction CPU can be :



Three address machine



Two Address machine



One address machine



Zero Address machine

DESING AND ORGANIZATION OF AC MACHINE

- ✓ The simplest way to organize a computer is to have one processor register(AC) and an instruction code format with two parts.

AC- machine has one processor register : AC(Accumulator)

- The operation is performed with the **memory operand** and the **content of AC**

Instruction code format with two parts : Op. Code + Address

- **Op. Code** : specify 16 possible operations(4 bit)
- **Address** : specify the address of an operand(12 bit)

- ✓ If an operation in an instruction code does not need an operand from memory, the rest of the bits in the instruction(address field) can be used for other information.
- ✓ Instructions without perand are like **Clear AC**, **Increment AC**, **Complement AC**, ...
- ✓ **CPU Memory** : is required to store instructions and hold data.
  - ✓ We decide to used 8KB rewritable memory.
  - ✓ Its characterstic discription 4K x 16 bit
  - ✓ 12 bit addressline = 4096 word(Instruction and Data are stored)
  - ✓ Store each instruction code(program) and operand (data) in 16-bit memory word

BASIC COMPUTER REGISTERS /AC-MACHINE

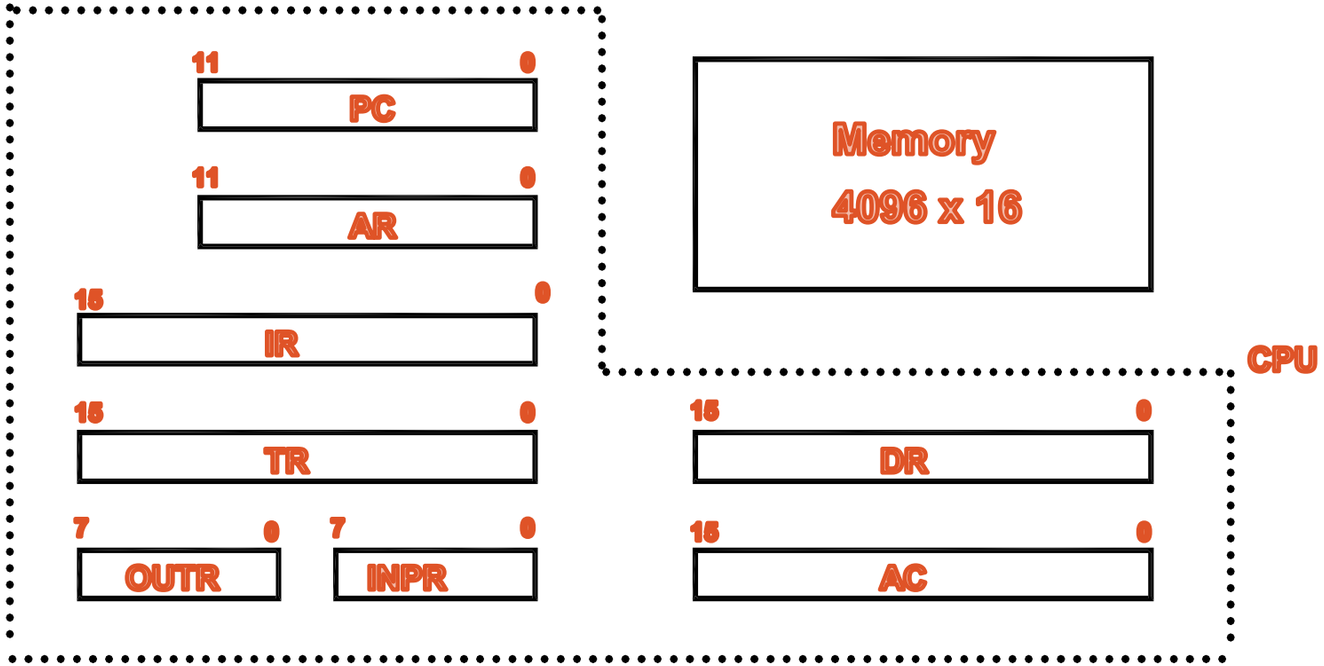
Computer instructions are normally stored in consecutive memory locations and are executed sequentially at a time.

List of Registers for the Basic Computer :

PROGRAM COUNTER(PC) : Hold the address of the next instruction to be read from memory after the current instruction is executed. Instruction words are read and executed in sequence unless a branch instruction is encountered. A branch instruction calls for a transfer to a non consecutive instruction in the program. The address part of a branch instruction is transferred to PC to become the address of the next instruction. To read instruction, memory read cycle is initiated, and PC is incremented by one.(next instruction fetch)

1. **DATA REGISTER(DR)** : hold the operand(Data) read from memory
2. **ACCUMULATOR REGISTER(AC)** : general purpose processing register
3. **INSTRUCTION REGISTER(IR)** : hold the instruction read from memory
4. **TEMPORARY REGISTER(TR)** : hold a temporary data during processing
5. **ADDRESS REGISTER(AR)** : hold a memory address, 12 bit width(in our case).
6. **INPUT REGISTER(INPR)** : receive an 8-bit character from input device
7. **OUTPUT REGISTER(OUTR)** : hold an 8-bit character for output device

BASIC COMPUTER REGISTERS



- Registers working with data are required to have the same size to the width of CPU memory. And registers
- working with address must have the same size to the Number of address line of CPU memory.
- Instruction register size is depending on number of operation the CPU supports and the CPU architecture.
- INPR and OUTR working with symbols coded by ASCII so are always 8 bit size in any CPU design.

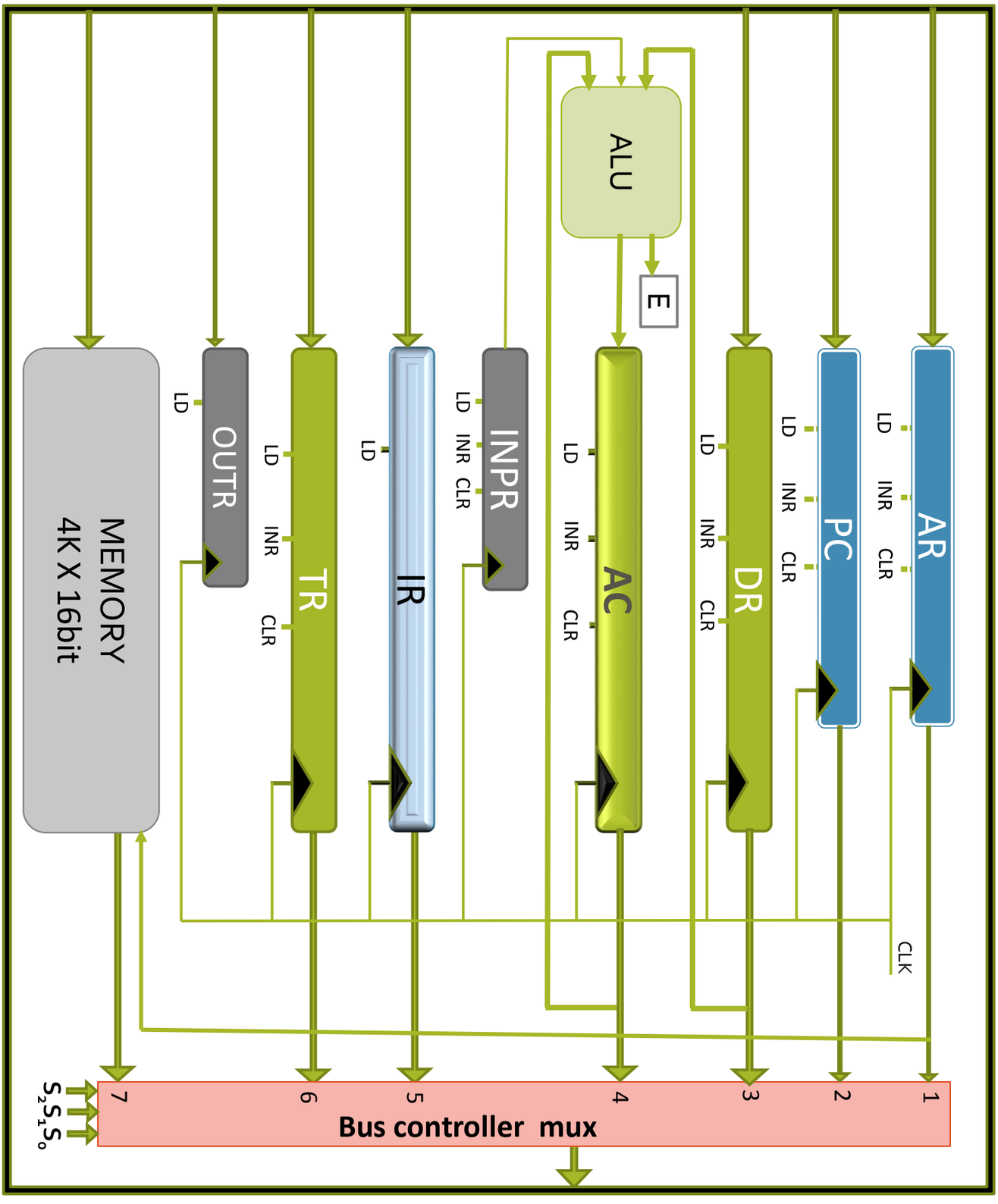
List of BC Registers

16	DR	Data Register
12	AR	Address Register
16	AC	Accumulator
16	IR	Instruction Register
12	PC	Program Counter
16	TR	Temporary Register
8	INPR	Input Register
8	OUTR	Output Register

COMMON BUS SYSTEM OF BASIC COMPUTER

- The basic computer has eight registers, a memory unit, and a control unit.
- Paths must be provided to transfer information from one register to another and between memory and registers.
- The registers in the Basic Computer are connected using a bus.
- This gives a savings in circuitry over complete connections between registers the connection of registers and memory of the basic computer to a common bus system is shown below:
- The system contain
  - Multiplexers
  - Bus
  - Registers
  - Memory

COMMON BUS SYSTEM OF AC MACHINE



Selected output of a single device will be linked to the bus at a time. And out of many device, devices having active LD will receive it from the bus

Cont'd

- The outputs of seven registers and memory are connected to the common bus and the specific output is Selected by mux($S_2S_1S_0$) :
  AR(1), PC(2), DR(3), AC(4), IR(5), TR(6), Memory(7)
- The particular register receives the data from the bus when LD is Enabled.
- The memory place its 16-bits o/p onto bus when the read input is activated and $S_2S_1S_0$ =111. when $S_2S_1S_0$= 011,the o/p of DR is placed on bus.
- The 12-bit registers, AR and PC, have 0's loaded onto the bus in the high order 4 bit positions
- Control Inputs : LD, INC, CLR ,Write, Read

Accumulator(AC) :

A 16-bits I/p of AC is from ALU. This register receives 3 inputs.

i. From o/p of ALU itself: To clear AC, to shift AC,…
ii. From DR : to ADD DR to AC, to AND DR to AC and so on…
iii. From INPR : to save or to process it.

Note: two micro operations can be executed at same time.

- DR ← TR : $S_2S_1S_0$= 110 (6) and LD of DR are required to be active.
- AC ← DR : DR ⟶ ALU ⟶ AC i.e. This is done by placing contents of DR on bus with $S_2S_1S_0$= 011 and enabling the LD of AC. The transfer happens at the end of clock pulse.

STORED PROGRAM ORGANIZATION OF AC MACHINE

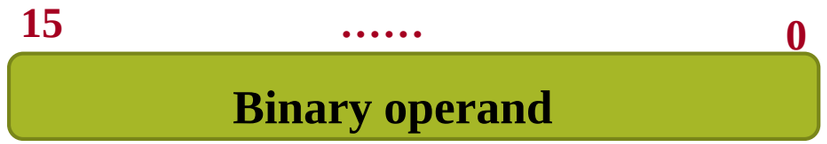
CPU memory has two parts program part called instruction part and data part

Instruction part: the part where instructions are located.

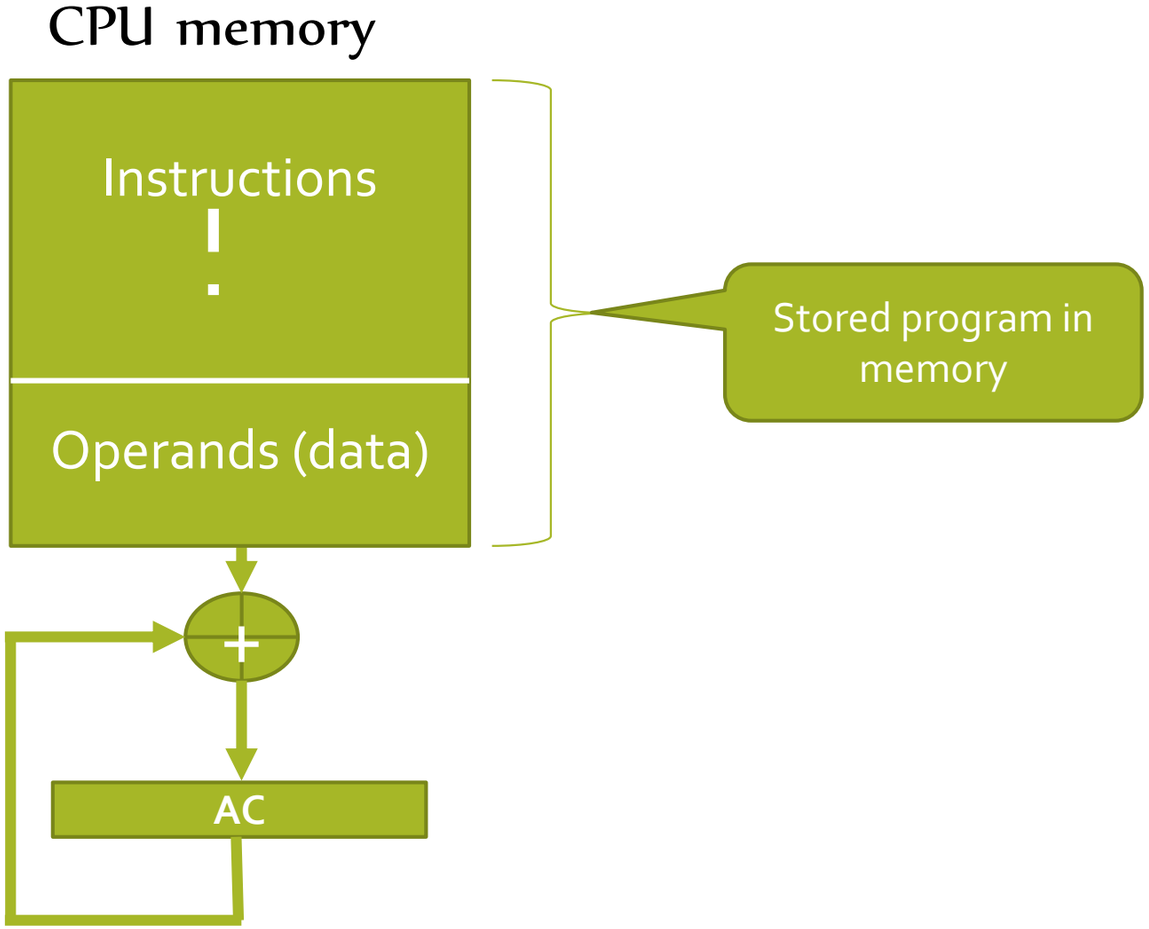
Data part: this is a location reserved for operands and address variables. A place where variables can be changed and strode on process



Instruction Format



Data register



ADDRESSING MODE

Immediate operand address :

the second part of an instruction code(address field) specifies an operand Address part used as actual operand e.g. R1←777

Direct operand address :

The second part of an instruction code specifies the address of an operand

e.g. R1←M[AR]

Indirect operand address :

The bits in the second part of the instruction designate an address of a memory word in which the address of the operand is found.

One bit of the instruction code is used to distinguish between a

direct and an indirect address .

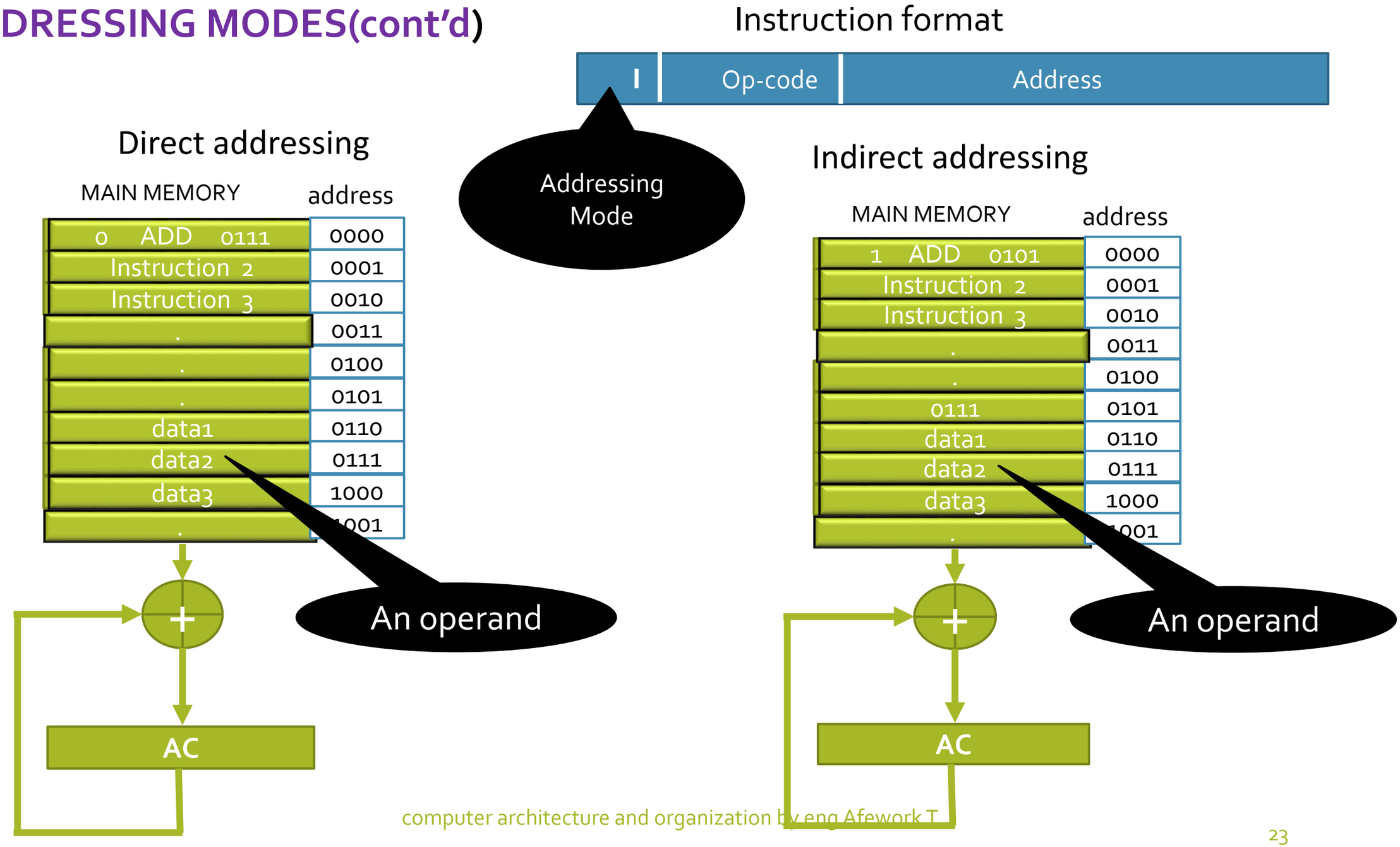
e.g. R1←M[M[AR]]

Effective Address

The operand address in computation-type instruction or the

target address in a branch-type instruction

ADDRESSING MODES(cont'd)

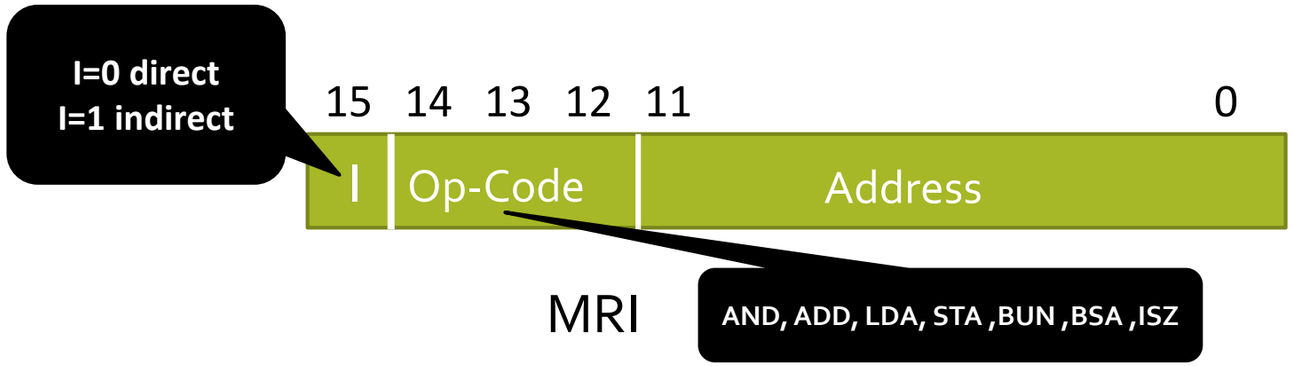


4-3 AC MACHINE INSTRUCTIONS

The basic computer has 3 Instruction Code Formats.
Each formats has 16-bits.

Memory-reference instruction:

- Operation which require operand from memory.
- Op-code part has 3-bits and indicate what kind of operation is performed
- The address part has 12-bits and indicate the address of operand.
- **I** is the addressing mode, if **I**=0,Direct address and if **I**=1,indirect address
- Op-code = 000 ~ 110, **I**=0 or **I**=1



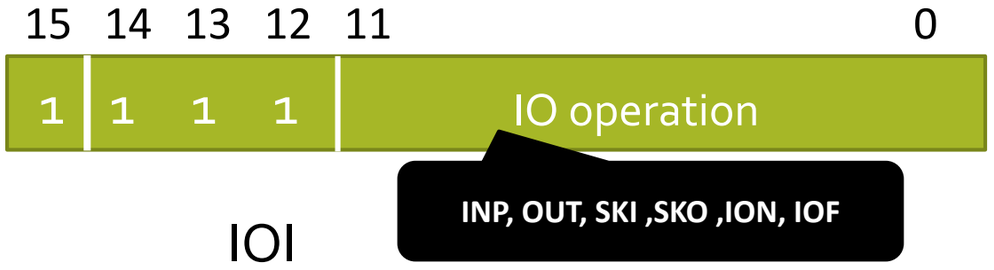
Register-reference instruction:

- Operations that can be done on register, require register operand
- If the Op-code =111 and I=0,then it is RRI.
- are operation on the AC register: e.g.:- : Clear AC, Complement AC,…..
- The 12-bits are used to specify the operation to be executed. E.g.:-CLA=7800



Input-Output instruction :

- Op-code=111 with I=1



- The 12-bits specify the i/o operation: INP, OUT, …. E.g.:-INP=F800 =1111 1000 0000 0000 = input a character to AC

COMPUTER INSTRUCTIONS(Cont'd)

The type of instruction is recognized by the computer control unit from 4-bits in position 12 through 15

E.g.:- If Op-code is not 111, then it is MRI

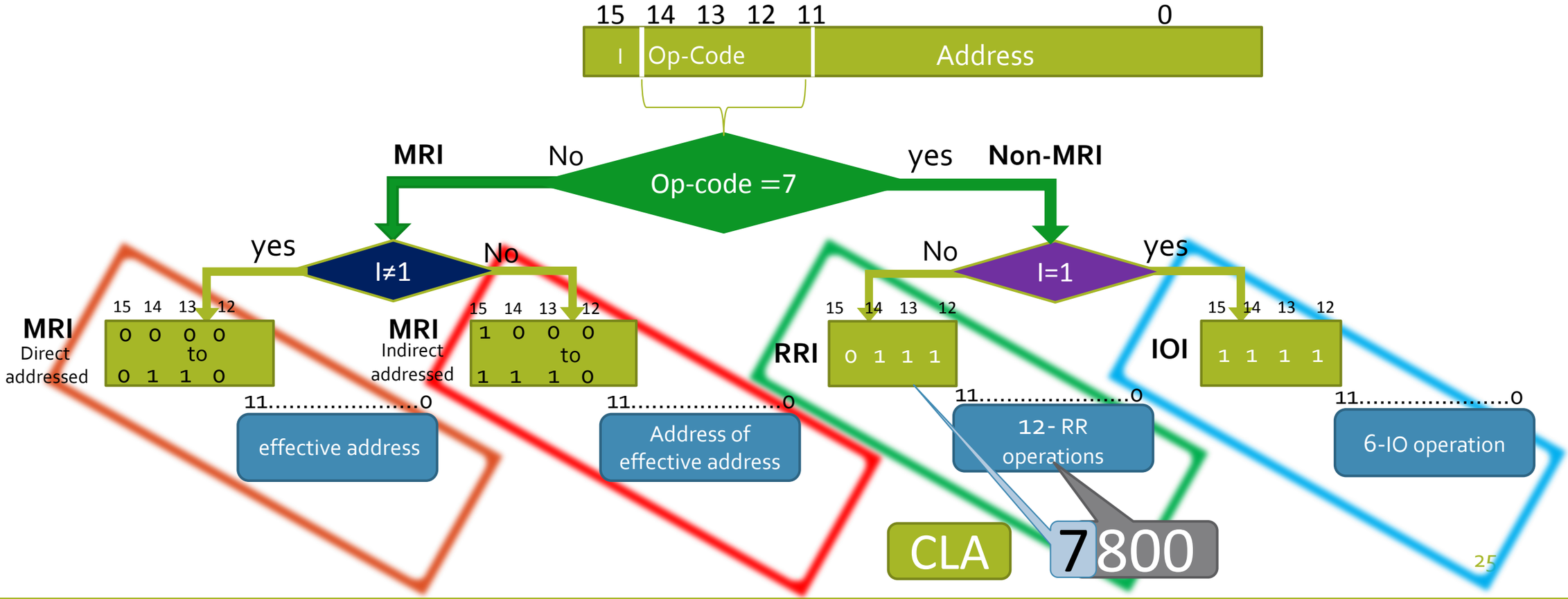
If Op-code=111 and I=0,then it is RRI

If Op-code=111 and I=1, then it is IOI

we have seen that only 3-bit are used for OP-CODE.

However, since RRI&IOI use the remaining 12-bits for operation,

the number of instruction will exceed 8. In fact, we have about 25 instruction in Basic computer.



BASIC COMPUTER INSTRUCTIONS

Symbol	Hex Code I = 0	Hex Code I = 1	Description
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load AC from memory
STA	3xxx	Bxxx	Store content of AC into memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instr. if AC is positive
SNA	7008		Skip next instr. if AC is negative
SZA	7004		Skip next instr. if AC is zero
SZE	7002		Skip next instr. if E is zero
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

MRI

RRI

IOI

INSTRUCTION SET COMPLETENESS

A computer should have a set of instructions so that the user can develop machine language programs to evaluate any function that is known to be computable. The set of instructions are said to be complete if the computer include all the following instruction :

- ✓ **Functional (Computational) Instructions**

  Arithmetic, logic, and shift instructions

  ADD, CMA, INC, CIR, CIL, AND, CLA

- ✓ **Transfer Instructions**

Data transfers between the main memory and the process registers

  LDA, STA

- ✓ **Control Instructions**

Program sequencing and control

  BUN, BSA, ISZ

- ✓ **Input/output Instructions**

Input and output

  INP, OUT

4-4 TIMING AND CONTROL

Clock pulses

- A master clock generator controls the timing for all registers in the basic computer.
- The clock pulses are applied to all FFs and registers in the system.
- The clock pulses do not change the state of a register unless the register is enabled by a necessary control signal/signals.
- The control signals are generated in the control unit.
- The control signals provide control inputs for the multiplexers ($S_2S_1S_0$) in the common bus, control inputs(INR,CLR,LD) in processor registers, and micro operations for the accumulator.

For completeness of control it uses combination of both control ways:

- Micro programmed control
- Hardwired control

The control which are uncovered by hardwired will be covered through programmed control.

Cont'd.

Micro programmed Control:

The control information is stored in a control memory, and the control memory is programmed to initiate the required sequence of micro operations.

Any required change can be done by updating the micro program in control memory(advantage)

Slow operation (disadvantage)

A control which is complete has a property to cover uncovered control in h/w type by s/w control type.

Hardwired Control:

The control logic is implemented with gates, F/Fs, decoders, and other digital circuits

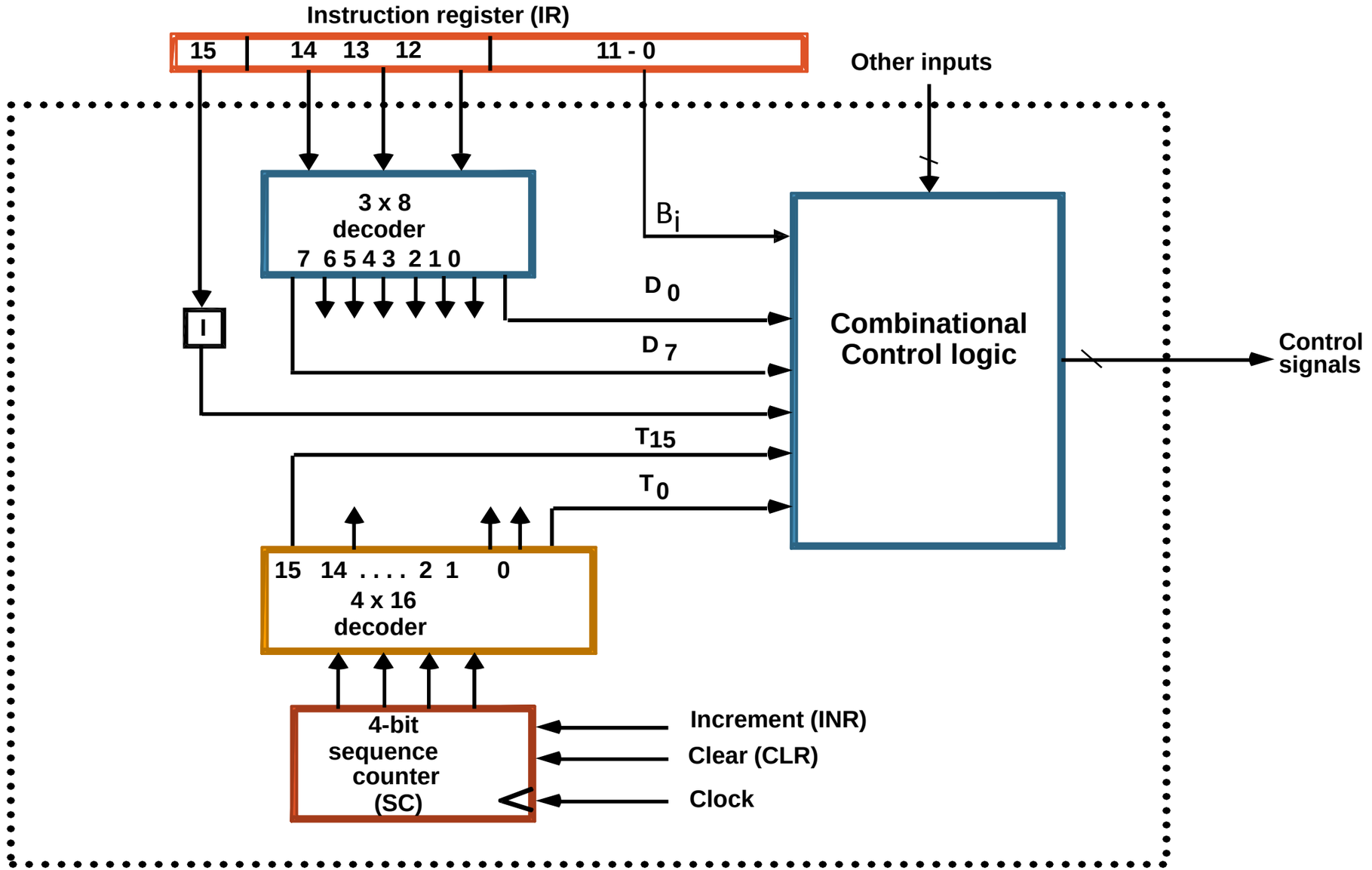
The operation is fast (advantage)

The Wiring change has to be occur if the design has to be modified (disadvantage)

Timing and Hard Wired Control

Control unit of basic computer/ AC machine

B_i = IR(i) , i=0,1,2,...,11(B_i specify the operation for RRI)

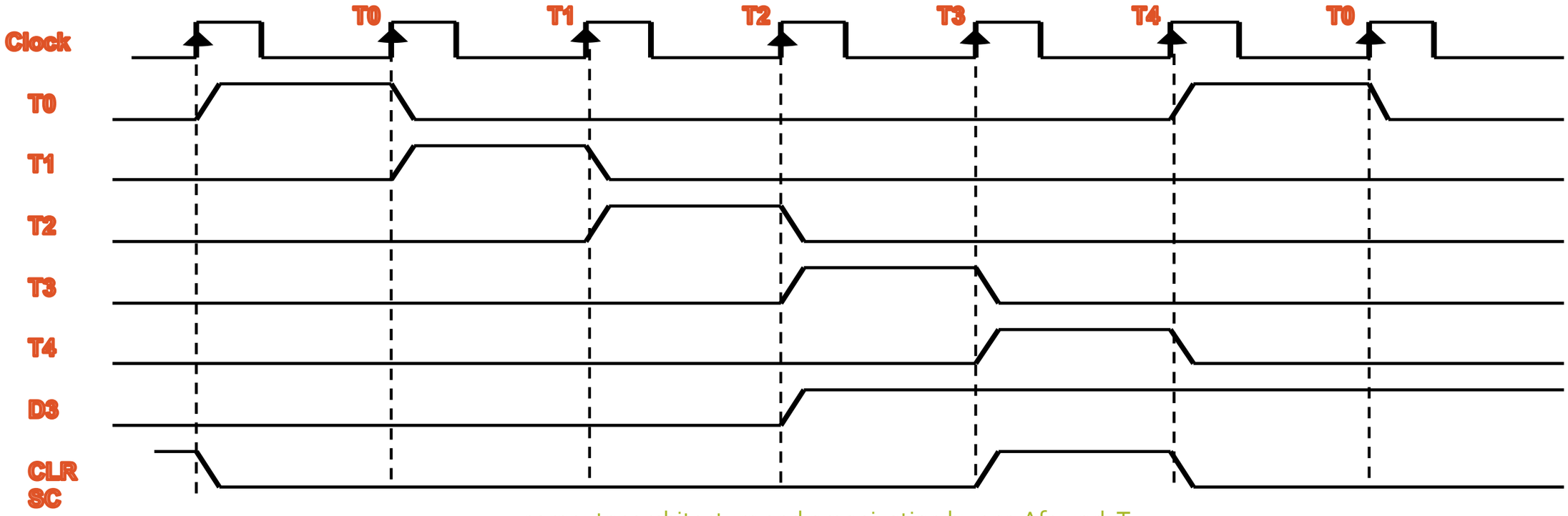


Operation of control unit

- ✓ Control unit = control logic gate + 3x4 decoder + instruction register + timing signal
- ✓ Timing control signal is generated by 4-bit sequence counter and 4×16 decoder.
- ✓ The instruction read from memory is placed in IR and op-code part is decoded by 3x8 decoder from it.
- ✓ The 4-bits SC(sequence counter) is decoded to 16 timing signals $(T_0 - T_{15})$.
- ✓ The SC can be incremented or cleared synchronously but, most time, the counter is incremented to provide the sequence of timing signals.
- ✓ Once the SC is cleared, this cause the next active timing signals to be T_0.

Operation of control unit (cont'd)

Let see the following example.
Consider the case where SC is incremented to provide timing signals T0,T1,T2,T3 &T4 in sequences.
Assume: At time T_4, SC is cleared to 0 if decoder output D_3 is active.
This is expressed symbolically by statement D_3T_4: SC ← 0
The timing diagram which show the relationship of the control signal is as below:



Operation of control unit (cont'd)

- ✓ The SC is respond to the edge of clock pulse. At the start the CLR input of SC is active.
- ✓ The first edge of clock pulse clears SC to 0,w/c in turn activates the timing signal T0.T0 is active for only one clock cycle.T0 will trigger only those registers whose control inputs are connected to timing signal T0.
- ✓ SC is incremented with every clock inputs pulse, unless its CLR i/p is active and this produce T0,T1,T2,T3 and T4. If SC is not cleared, the timing signals will continues with T5,T6,….T15 and back to T0.
- ✓ At T4,SC cleared and backed to T0.
- ✓ The last three waveforms shows how SC is cleared

  when : D3T4=1, D3T4: SC ← 0
- ✓ When T4 is active,D3T4=1 at the end of T4 and this cause T0 active.
- ✓ A memory read or write cycle will be initiated with the rising edge of a timing signals.

Operation of control unit (cont'd)

- ✓ It will be assumed that , a memory cycle time is less than the clock cycle time,
- ✓ According to this assumption, a memory read or write cycle time will be completed by the time next clock goes through its positive transition.
- ✓ The clock transition will then be used to load the memory word to a register.

  Example:- Register transfer statement :

  $$T0 : AR \leftarrow PC$$

- ✓ This specifies a transfer of the content of PC into AR if timing signal T0 =1 ,T0 is active during entire clock cycle interval.
- ✓ During T0 active the content of PC is placed onto the bus (S2S1S0=010) LD(load) input of AR is enabled, the actual transfer occurs at the next positive transition of the clock(T0 rising edge clock)
- ✓ This same clock transition increments the sequence counter SC from 0000(T0) to 0001(T1). Sothat,T1 become active and T0 become inactive.

4-5 INSTRUCTION CYCLE

The program is executed in the computer by going through a cycle for each instruction. Each instruction cycle in turn is subdivided into a sequence of sub cycle or phases.

In Basic Computer, a machine instruction is executed in the following cycle:

1st.Fetch an instruction from memory

2nd.Decode the instruction

3rd.Called write back :- because memory access required here to read the effective address from memory if the instruction has an indirect address

4th.Execute the instruction

Go to step 1 : Next Instruction[PC + 1]

After an instruction is executed, the cycle starts again at step 1, for the next instruction. This process continue indefinitely unless HALT instruction is encountered.

Fetch and Decode instruction cycle

The micro operation that specifies the fetch and decode phase is shown as below :

T0: AR ← PC (S2S1S0= 010, T0=1)

T1: IR ← M [AR], PC ← PC + 1 (S2S1S0= 111, T1=1)

T2: D0, . . . , D7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(15)

fetch cycle

At T0 = 1, load the address of instruction (PC) onto AR.

T0: AR ← PC

Place the content of PC onto the bus by making the bus selection inputs

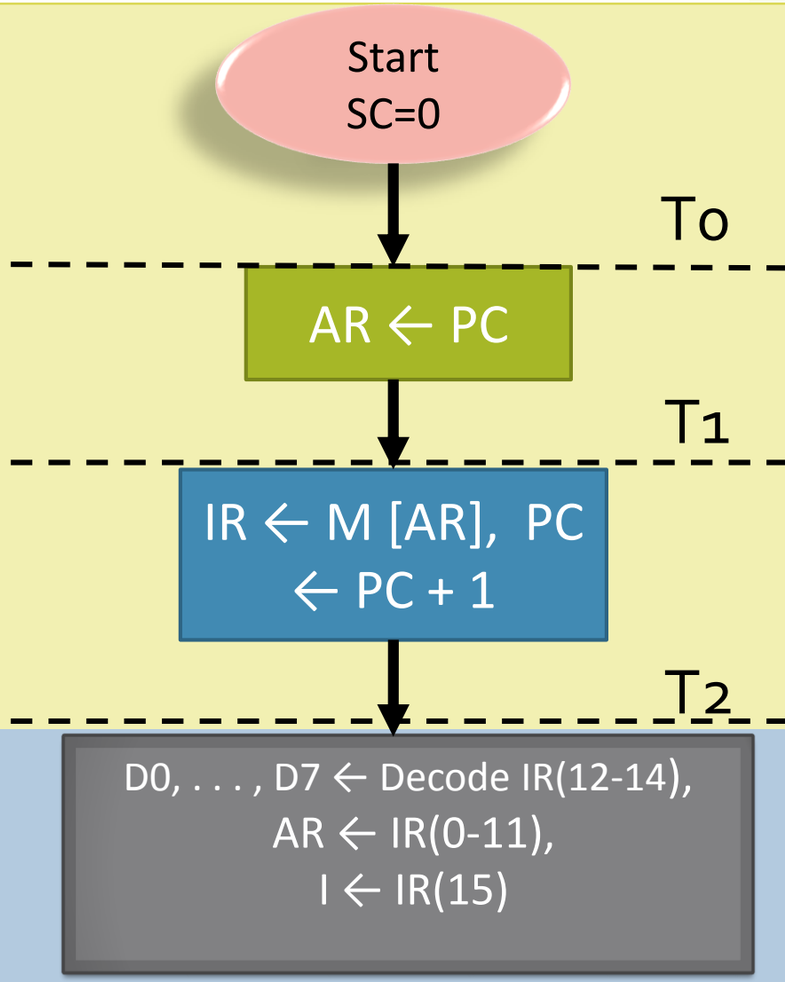
S2S1S0=010. Transfer the content of the bus to AR by enabling the LD input of AR. At T1 = 1, fetch instruction. From memory is transferred to IR and PC is incremented

T1: IR ← M [AR], PC ← PC + 1

Enable the read input memory, Place the content of memory onto the bus by making S2S1S0= 111

Transfer the content of the bus to IR by enable the LD input of I

Increment PC by enabling the INR input of PC.



Decode cycle

At T2=1,opcode is decoded the indirect bit transferred to I FF and address part transferred to AR

T2: D0, . . . , D7 Decode← IR(12-14), AR ← IR(0-11), I ← IR(15)

Note that SC is incremented at each clock pulse to provide T0,T1,….

computer architecture and organization by eng Afework.T

Fetch and Decode (cont'd)

Basic components used instruction fetch and decode are as shown

Example:

let see how the register transfer for the fetch is implemented in the bus system.

T0: AR ← PC (S0S1S2=010, T0=1)

T1: IR ← M [AR], PC ← PC + 1(S0S1S2=111,T1=1)

T0: AR ← PC (T0 = 1)

Bus← PC, by making S2S1S0=010

AR← BUS, by making LD =1.

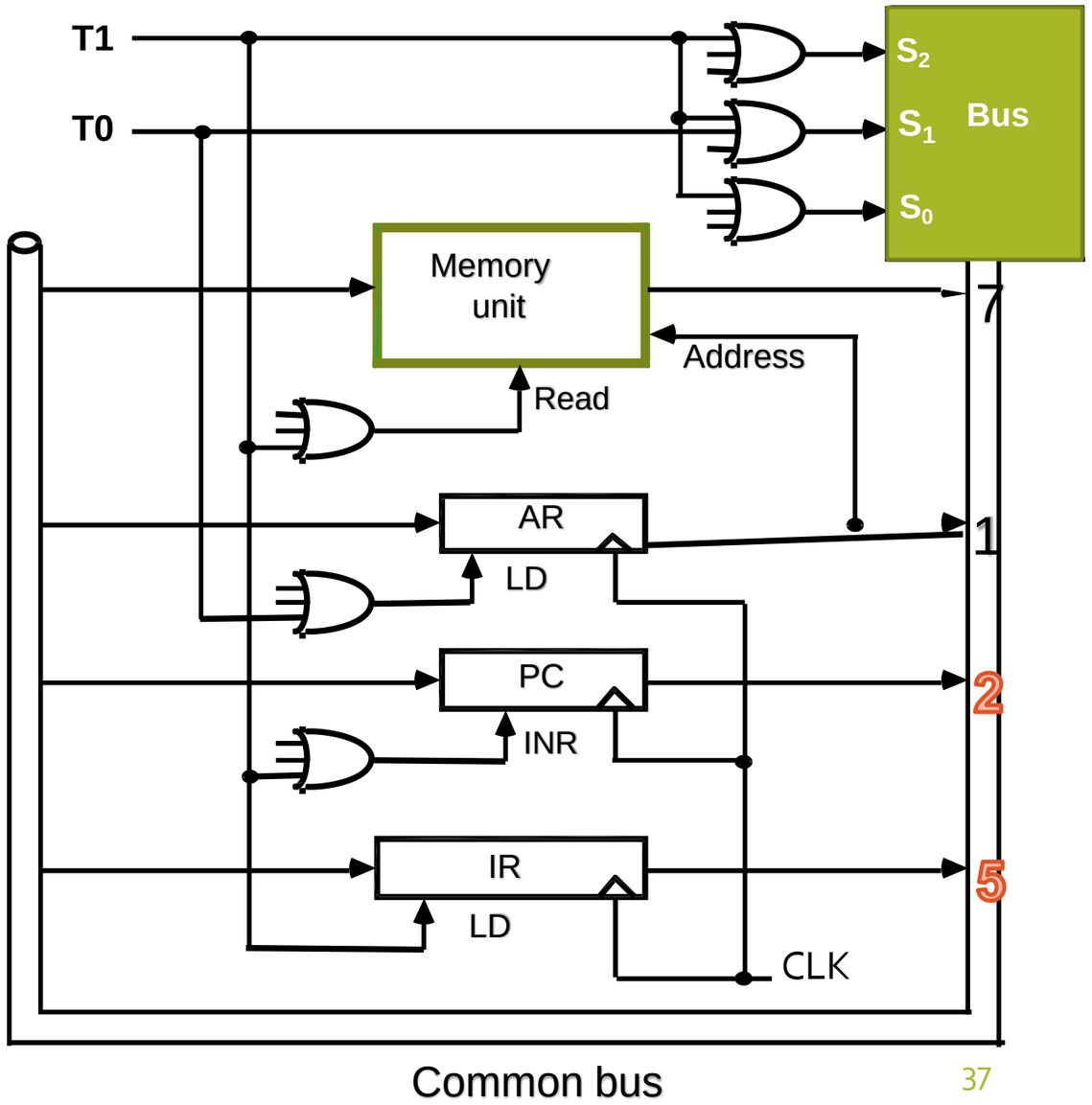
T1: IR ← M [AR], PC ← PC + 1(T1=1)

Make read=1

BUS ← M[AR], by making S2S1S0=111

IR ← BUS, by making LD=1

PC ← PC + 1,by making INR=1



DETERMINING THE TYPE OF INSTRUCTION

The timing signal that active after decoding is T3.

At T3=1:

During time T3, the control unit determine the type of the instruction that was just read from memory.

The following flow chart present an initial configuration for the instr. cycle and shows how the control determine the instruction types after decoding.

The three possible instruction types are

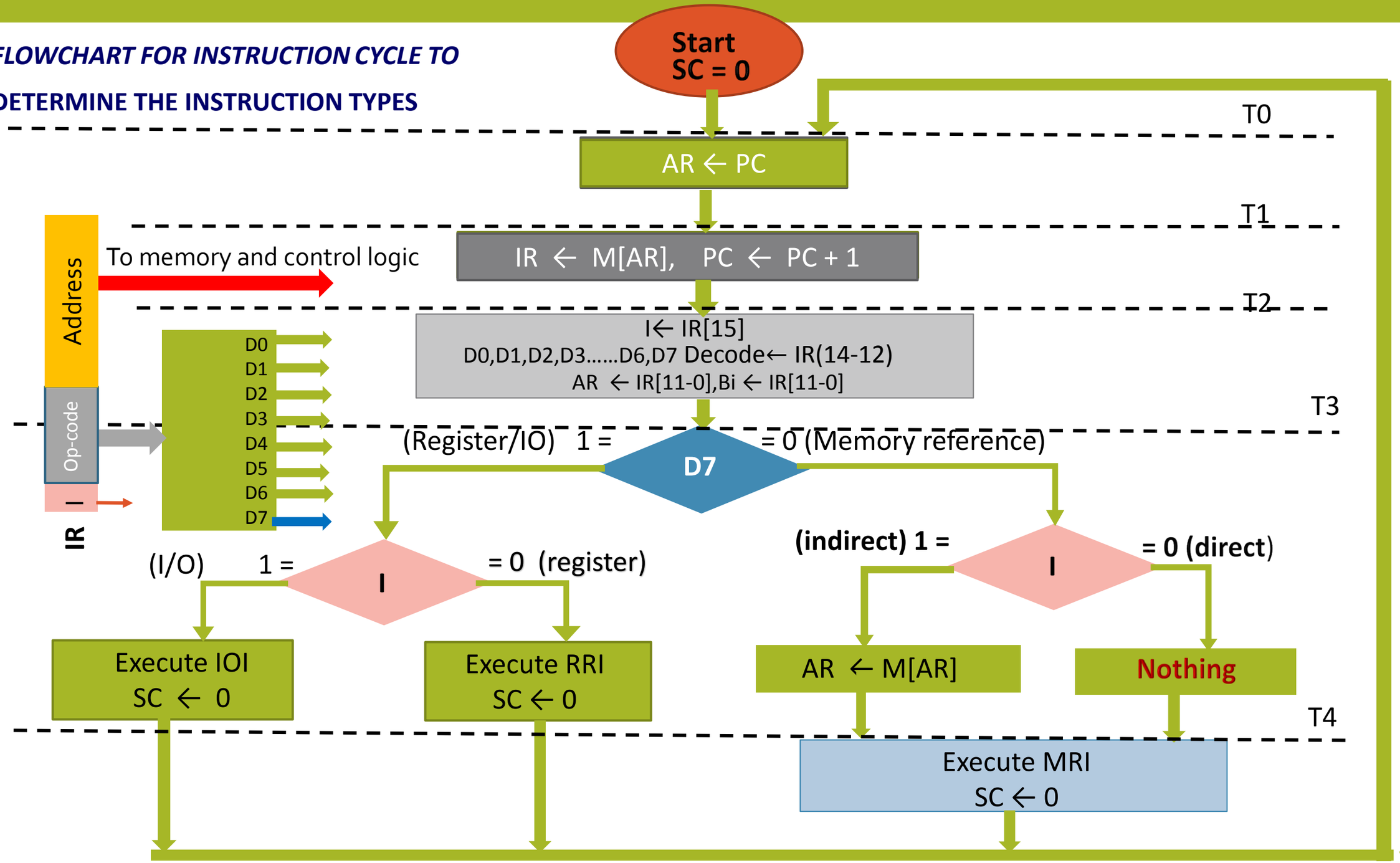
Memory-reference instruction

Register-reference instruction and

Input-output instr.

FLOWCHART FOR INSTRUCTION CYCLE TO

DETERMINE THE INSTRUCTION TYPES



DETERMINE THE INSTRUCTION TYPES (*cont'd*)

Decoder out put D_7=1, when op-code=111.
If D_7=1 then the instruction type could be either **RRI or IOI** based on the bit of I.
If D_7=0,then the instr. is **MRI** type then, the control inspect the value available in **I** FF to determine direct or indirect used addressing mode.
The three-instruction types are subdivided into 4-separate paths.
The selected operation is activated with the clock transition associated with timing signal T3.
This can be symbolized as follows

D_7'I'T3: AR ← M[AR]
D_7'IT3:Nothing
D_7I'T3: Execute a register-reference instr.
D7IT3: Execute an input-output instruction.

When MR instruction with I=0, nothing done since the EA is already in
AR (AR ← IR(0-11))
However, SC must be incremented to provide T4.

REGISTER REFERENCE INSTRUCTIONS

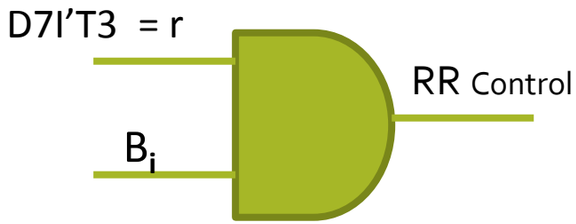
Register Reference Instructions are identified when $D_7 = 1$, $I = 0$

The operation of Register Ref. Instr. is specified in $B_0 \sim B_{11}$ of IR execution starts with timing signal T_3

Each control function need , $D_7\, I'T_3$,w/c designated by r. since $D_7 = 1$, I=0 AND T3 =1.

$r = D_7\, I'T_3$ => Register Reference Instruction B_i = IR(i) , i=0,1,2,...,11(B_i specify the operation for RRI)

	r:	SC ← 0
CLA	rB_{11}:	AC ← 0
CLE	rB_{10}:	E ← 0
CMA	rB_9:	AC ← AC'
CME	rB_8:	E ← E'
CIR	rB_7:	AC ← shr AC, AC(15) ← E, E ← AC(0)
CIL	rB_6:	AC ← shl AC, AC(0) ← E, E ← AC(15)
INC	rB_5:	AC ← AC + 1
SPA	rB_4:	if (AC(15) = 0) then (PC ← PC+1)
SNA	rB_3:	if (AC(15) = 1) then (PC ← PC+1)
SZA	rB_2:	if (AC = 0) then (PC ← PC+1)
SZE	rB_1:	if (E = 0) then (PC ← PC+1)
HLT	rB_0:	S ← 0 (S is a start-stop flip-flop)



REGISTER REFERENCE INSTRUCTIONS(cont'd)

RRI and IOI executed at T3

After the execution, SC cleared and the control return to fetch phase with To=1.

When SC is to be cleared , we include the statement SC ← 0

Example:

CLA has the hexadecimal code 7800, w/c gives 0111 1000 0000 0000

CLA=rB11: $D_7I'T_3B_{11}$ =7800 = AC ← 0

- The first bit is 0, so that I=0 , w/c is I'. The next 3-bits are 111, so that D7 is equal to 1. Only one bit at 11th. Position is 1, B11.
- The control function that initiate the micro operation for this instruction is $D_7I'T_3B_{11}$

4.6 MEMORY REFERENCE INSTRUCTIONS

The seven MRI are listed below.

Symbol	Operation Decoder	Symbolic Description
AND	D_0	**AC ← AC ∧ M[AR]**
ADD	D_1	**AC ← AC + M[AR], E ← C_{out}**
LDA	D_2	**AC ← M[AR]**
STA	D_3	**M[AR] ← AC**
BUN	D_4	**PC ← AR**
BSA	D_5	**M[AR] ← PC, PC ← AR + 1**
ISZ	D_6	**M[AR] ← M[AR] + 1, if M[AR] + 1 = 0 then PC ← PC+1**

- The o/p D_i is come from opcode decoder.
- The effective address of the instruction is in AR and was placed there during timing signal T2 when I = 0, or during timing signal T3 when I =1
- Memory cycle is assumed to be short enough to complete in a CPU cycle.
- The execution of MR instruction starts with T_4.
- The symbolic description is specified in above table is in terms of RT notation.

THE CONTROL FUNCTION AND MICRO OPERATION NEEDED FOR EXECUTION OF EACH MRI

1. **AND to AC** : Micro operations needed to execute this instructions are:

   D_0T_4: DR ← M[AR] Read operand

   D_0T_5: AC ← AC ∧ DR, SC ← 0 AND with AC

2. **ADD to AC**: Micro operations needed to execute this instructions are:

   D_1T_4 : DR ← M[AR] Read operand

   D_1T_5 : AC ← AC + DR, E ← C_{out} , SC ← 0 Add to AC and store carry in E

3. **LDA**: **Load to AC** : Micro operations needed to execute this instructions are:

   D_2T_4 :DR ← M[AR] Read operand

   D_2T_5 : AC ← DR, SC ← 0 Load to AC

   Since, there is no direct connection of AC with Bus, AC must receive from DR

4. **STA: Store AC**: Micro operations needed to execute this instructions are:

   D_3T_4 : M[AR] ← AC, SC ← 0 Store AC in a memory

THE CONTROL FUNCTION AND MICRO OPERATION NEEDED FOR EXECUTION OF EACH MRI(cont'd)

5. BUN: Branch Unconditionally

The BUN instruction allows the programmer to specify an instruction out of sequence and we say that the program branches (or jumps) unconditionally. Micro operations needed to execute this instructions are:

D_4T_4 : PC ← AR, SC ← 0: The EA (AR) is transferred to PC.

6. BSA: Branch and Save Return Address

This instruction is useful for branching to a portion of the program called a subroutine or procedure.

When executed, the BSA instruction stores the address of the next instruction in sequence (w/c is available in PC) into a memory specified by EA(AR). Then AR+1 transferred to pc to serve as the address of first instruction in the Subroutine.

D_5T_4 : M[AR] ← PC, AR ← AR + 1

D_5T_5 : PC ← AR , SC ← 0

THE CONTROL FUNCTION AND MICROOPERATION NEEDED FOR EXECUTION OF EACH MRI(cont'd)

7. ISZ: Increment and Skip-if-Zero

This instruction increment the word specified by EA and if the incremented value is equal to 0, PC incremented by one.

D_6T_4 : DR ← M[AR]

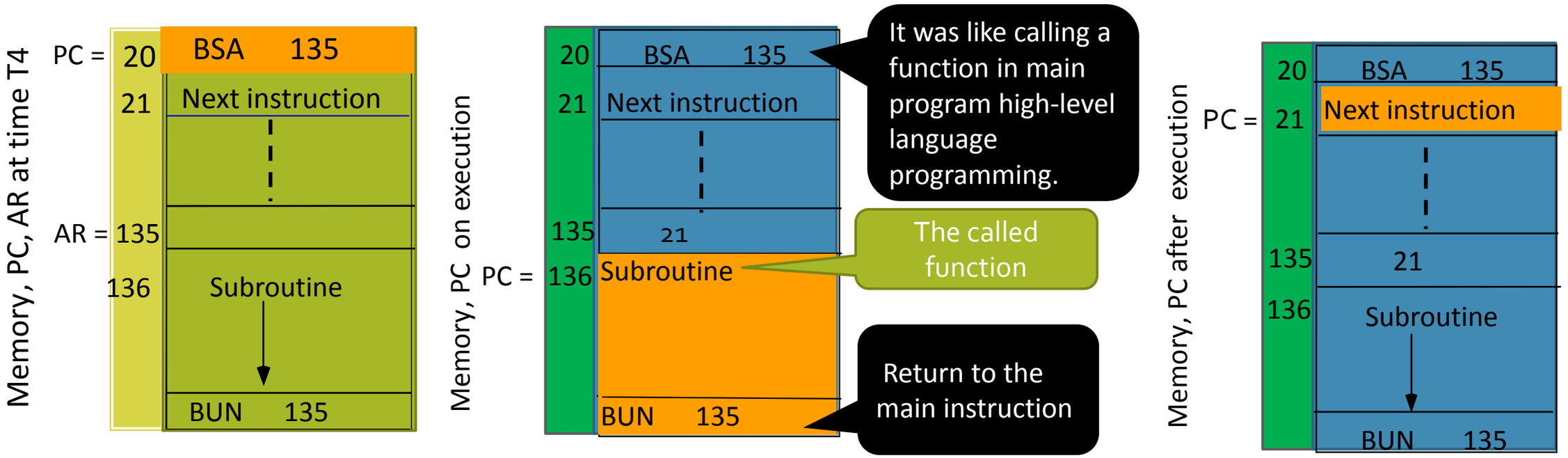
D_6T_5 : DR ← DR + 1

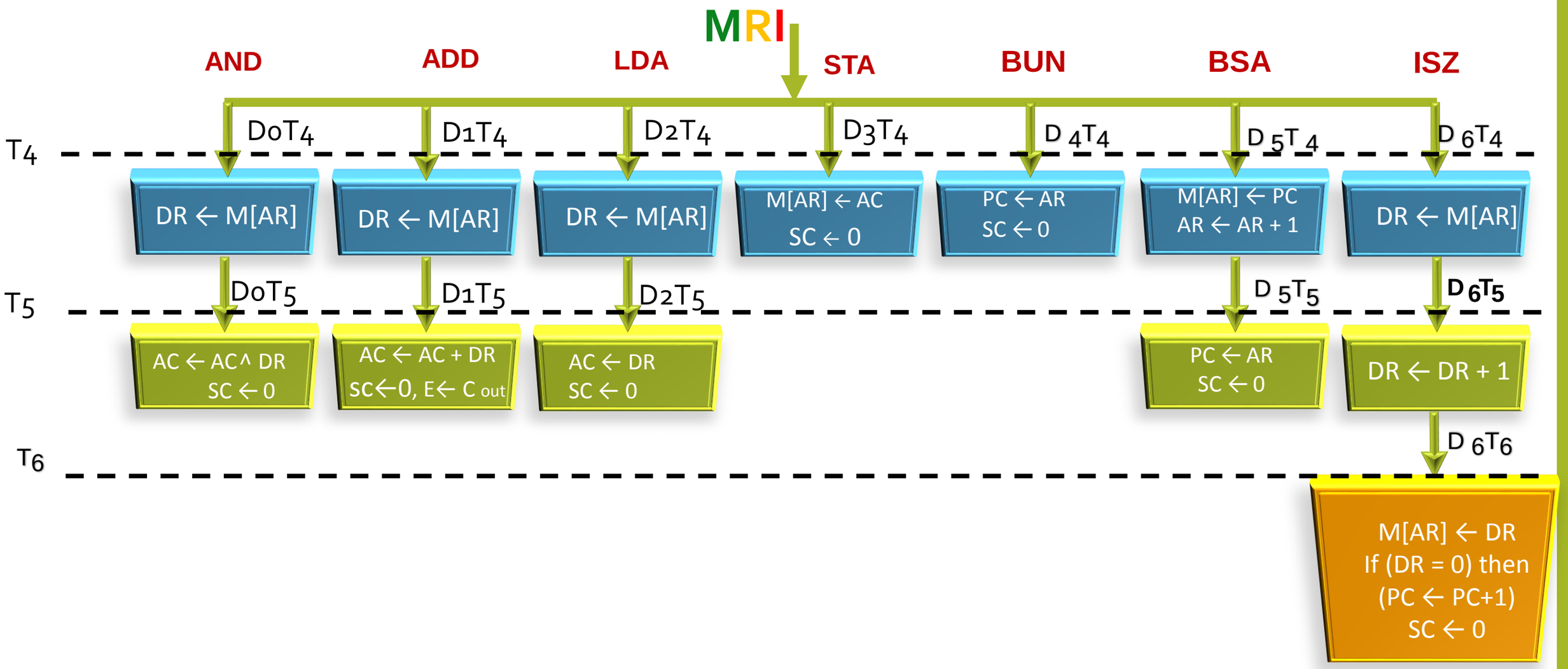
D_6T_6 : M[AR] ← DR, if (DR = 0) then (PC ← PC + 1), SC ← 0

if (DR ≠ 0) then (PC ← PC), SC ← 0

EXAMPLE OF BRANCH AND SAVE ADDRESS INSTRUCTION

let BSA instruction is at address of 20 and address part has 135, After fetch and decode PC become PC+1=21, the return address(21),AR hold EA=135,so that, BSA perform M[AR] ← 21,
AR ← 135 + 1=136.
Here, save the return address(21) in AR=135
The next instruction address will be PC ← PC + 1=21
The return to original program(to PC=21) is done by means of indirect BUN instruction at the end of subroutine.
When BUN instruction is executed, the Effective Address 21 is transferred to PC indirectly.



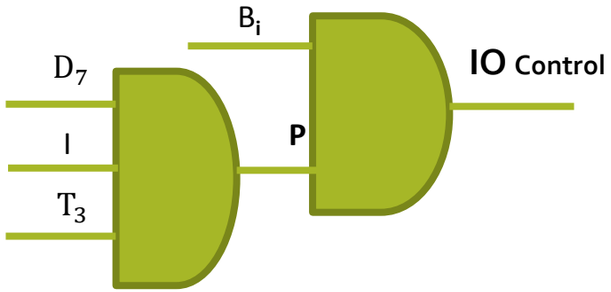

MRI
AND
ADD
LDA
STA
BUN
BSA
ISZ
DoT4
D1T4
D2T4
D3T4
D 4T4
D 5T 4
D 6T4
T4
DR ← M[AR]
DR ← M[AR]
DR ← M[AR]
M[AR] ← AC
SC ← 0
PC ← AR
SC ← 0
M[AR] ← PC
AR ← AR + 1
DR ← M[AR]
DoT5
D1T5
D2T5
D 5T5
D 6T5
T5
AC ← AC^ DR
SC ← 0
AC ← AC + DR
SC←0, E← C out
AC ← DR
SC ← 0
PC ← AR
SC ← 0
DR ← DR + 1
D 6T6
T6
M[AR] ← DR
If (DR = 0) then
(PC ← PC+1)
SC ← 0

Input-Output Instructions

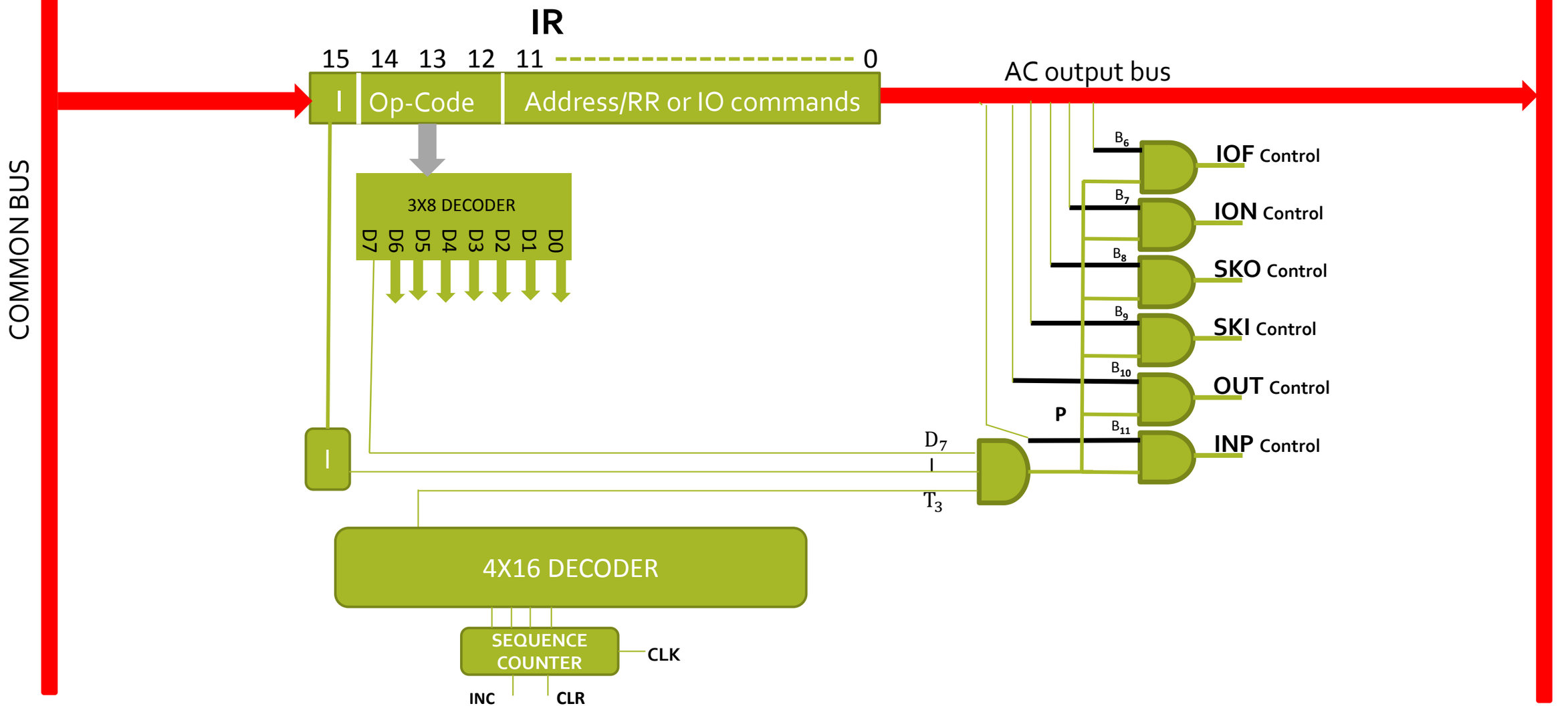
- Input and output instructions are needed for transferring information to and from *AC* register, for checking the flag bits, and for controlling the interrupt facility.
- Input-output instructions have an operation code 1111 and are recognized by the control when $D_7 = 1$ and $I = 1$. The remaining bits of the instruction specify the particular operation.
- The control functions and microoperations for the input-output instructions are executed with the clock transition associated with timing signal T3 .
- Each control function needs a Boolean relation D_7IT3, which we designate for convenience by the symbol *p*. The control function is distinguished by one of the bits in IR(6-11). By assigning the symbol *B;* to bit *i* of *IR,* all control functions.
- $D_7IT_3 = P$ (common to all input-output instructions)

$IR_i = B_i$ [bit in IR[11:6] that specifies the instruction]

	P:	SC ← 0	Clear SC
IOF	PB_6:	IEN ← 0	Interrupt enable off
ION	PB_7:	IEN ← 1	Interrupt enable on
SKO	PB_8:	If (FGO = 1) then (PC ← PC + 1)	Skip on output flag
SKI	PB_9:	If (FGI = 1) then (PC ← PC + 1)	Skip on input flag
OUT	PB_{10}:	OUTR ← AC(0-7), FGO← 0	Output character
INP	PB_{11}:	AC(0-7) ← INPR, FGI ← 0	Input character



IOI hardwired control



READING ASSIGNMENT: PROGRAM INTERRUPT

Summery

- On this chapter we studied
  - ✓ AC-architecture and organization
    - (CPU organization) ALU configuration and component specifications using 4096x16bit memory and to implement 16 operations(Who can list them?)
    - 8 basic registers are required where a lot of microoperations are done on AC register.
    - AC is default destination for a lot of microoperations
    - 16 bit IR is required to hold the instruction when fetched.
    - For convenient control instructions are
      - MRI,RRI,IOI
    - The control unit will identify the way in which they are executed and determines the cycle time by its sequence counter.
  - ✓ operation of control unit
      - All of its guiding input are from IR and helping inputs from sequence counter and other external inputs from external devices and flag registers.
      - Master clk is feed to it for its operation.
      - It generates control signals to the required device according to the instruction identity

Instruction cycle in AC-Architecture has four stages:

IF ID EA Ex

The number of stage depends on the type of architecture the CPU is organized.

In any architecture the maximum size of instruction register must not be greater than the word size of memory. Short instructions are faster to be executed.

Summery questions:

1. What is an accumulator machine?
2. A CPU made of 64kB internal memory in AC-ISA ,to implement 16 operation supported by 32bit common bus system. what are the size for each registers?
3. What is psuedoinstruction?
4. What is instruction cycle?
5. What are flag registers in basic computer design?
6. What is instruction cycle time?
7. When do instruction cycle can get timing signal greater than T4? Discuss it with parables. The timing is called cycle time. Which instruction cycle can take large cycle time?
8. At what instruction cycle do we need to clear SC? And Why? What if not cleared?
9. Describe the importance and size of registers named I,E,R,FGO,FGI,S and IEN .
10. Why do we need to program interrupts?

